



Little IR12660's Adventure Contest Editorial

OCPC 2025 Winter, Contest Day 6
By: tibinyte2006, ntherner



A

Given is array a

Partition into A and B s.t. $f(A) + f(B) \rightarrow \max$

What is $f(A)$?

$f(A)$:

- Sort A $\downarrow$
- return $\max(A_i \cdot i)$

Let's define function g

$g(A, i)$:

- Sort A $\downarrow$
- return $A_i \cdot i$

it follows that $\max f(A) + f(B) = \max g(A, i) + g(B, j)$

Back to the problem:

Sort array a : $a_1 \leq a_2 \leq \dots \leq a_m$

Fix i and j : $a_1 \leq a_2 \leq \dots \leq \underline{a_i} \leq a_{i+1} \leq \dots \leq a_{j-1} \leq \underline{a_j} \leq a_{j+1} \leq \dots \leq a_m$

$\underbrace{\hspace{15em}}_A \qquad \underbrace{\hspace{15em}}_B$

it is optimal to take A and B as shown

Thus: $ans = \max_{i < j} (j-i) a_i + (m-j+1) a_j$

Complexity: $O(m^2)$, can we optimize?

Likho!!

Fix j , find best $i \Leftrightarrow$ find $\max_i (j-i) a_i$

$(j-i) a_i = j a_i - i a_i$, which is a linear function $\alpha x + \beta$ with

$$\alpha = a_i, \beta = -i a_i$$

We can use a LiChao tree to get $O(m \log C)$ time complexity

B

Given A , find B s.t. for all $1 \leq i \leq |A|$,

$$\text{mex}(B_1, B_2, \dots, B_i) \leq \text{mex}(A_1, A_2, \dots, A_i)$$

$$B \subseteq A$$

How to solve without the second constraint?

Let $dp(i > j > k)$ be the number of arrays B s.t.

- $|B| = i$
- $\text{mex}(B) = j$
- there are k elements $> j$ which we don't know yet

Let's consider adding some value x ; we have 3 cases:

- $x < j \Rightarrow$ nothing happens, j ways to choose x
- $x > j \Rightarrow k \rightarrow k+1$, 1 way (we don't know x yet)
- $x = j \Rightarrow j$ increases to some j' , we need to count ways to cover

$[j+1, j'-1]$ with some of the k unknowns. This is Stirling numbers of the 2nd kind.

Complexity: $O(n^3)$

Back to the problem

$B \leq A$ means the following

$$\begin{array}{ccccccc} A_1 & A_2 & A_3 & A_4 & \dots & A_k & A_{k+1} & \dots \\ | & | & | & | & & | & \vee & \\ B_1 & B_2 & B_3 & B_4 & \dots & B_k & B_{k+1} & \dots \end{array}$$

We can fix k and B_{k+1} and run the dp; complexity $O(m^2)$

To be 3 more cases:

- $B_{k+1} < \text{mex}$, $\text{ways} = \max(0, \text{mex} - A_{k+1} - 1)$, run dp normally
- $B_{k+1} = \text{mex}$, run dp normally
- $B_{k+1} > \text{mex}$, treat B_{k+1} as an unknown, but add a special flag for it

complexity: $O(m^6)$

C Given binary string S , perform minimum flips s.t.:

$$2. \sum_{i=l}^r S[i] \geq r-l+1, \forall 1 \leq l \leq r \leq n, r-l+1 \geq k$$

Solution

Greedy: Add characters one by one. If the constraint is violated, flip the current character.
How to check if the constraint is violated?

Build array A , $A[i] = \begin{cases} 1, & S[i] = 1 \\ -1, & S[i] = 0 \end{cases}$

$$r \text{ Acs} \geq 0 \Leftrightarrow P[r] - P[l-1] \geq 0 \quad (P \text{ is prefix sum})$$

Now, subarray $l \dots r$ is good $\Leftrightarrow \sum_{i=l}^r A[i] \geq 0$

All subarrays that can become bad after adding some character are suffixes.
So, we need to check whether a suffix is bad $\Leftrightarrow \exists y \geq 0$ s.t. $P[i] - P[y] < 0, i-y \geq k$

How to do it efficiently?

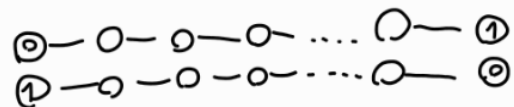
$$\min_{j=0}^{i-k} (P[i] - P[j]) = P[i] - \max_{j=0}^{i-k} (P[j])$$

So, we need max prefix sum and point updates (for when we append new chars)
We can solve this with a Segment Tree! Complexity: $O(n \log n)$,

D

Given tree, how many labelings with $(0, 1, 2, \dots, m-1)$ minimize sum of MEX over all paths?

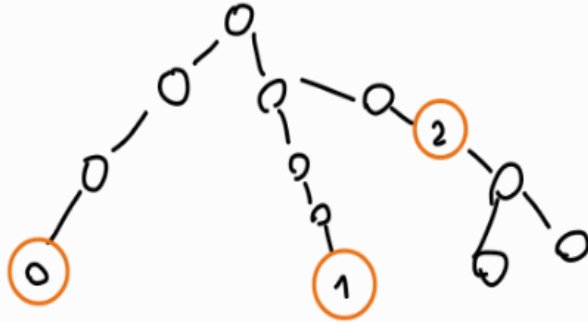
Have to solve for a chain?



We get a cost of $2m-1$; the number of ways is $2 \cdot (m-2)!$.

How to solve for the general case?

Place 0 and 1 in 2 leaves. Place 2 anywhere but not on the path.



$$\text{cost} = m-1 \text{ (we have } m-1 \text{ paths with mex} = 1) + 2 \text{ (we have one path with mex} = 2) = m+1$$

How to count efficiently?

$$ans = \sum_{(u,v) \in \text{leaves}} (m-1 - \text{dist}(u,v)) \cdot (m-3)! = (m-1) \cdot (m-3)! - (m-3)! \cdot \sum_{(u,v) \in \text{leaves}} \text{dist}(u,v)$$

To compute $\sum \text{dist}(u,v)$ we count contribution of each edge



if $\bullet$ has a leaves and $\bullet$ has b , the edge contributes in $a \cdot b$ paths!

Total complexity: $O(m)$

E

Count permutations of $\{1, 2, \dots, n\}$ with m restrictions:

1: subarray $l \dots r$ is a mountain 

2: subarray $l \dots r$ is a valley 

Solution

Subarray $l \dots r$ is a mountain $\Leftrightarrow$

- $a[l] < a[l+1]$
- $a[r] < a[r-1]$
- $\nexists i$ s.t. $a[i-1] > a[i] < a[i+1]$ (no local minimum)

So, the m restrictions are essentially restrictions of the kind:

- $a[i] < a[i+1]$
- $a[i] > a[i+1]$
- $a[i]$ cannot be local minima
- $a[i]$ cannot be local maxima

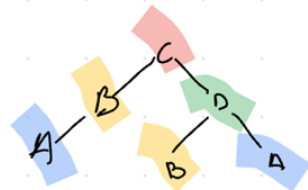
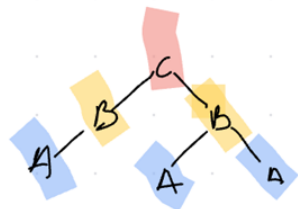
let $dp(i, y, flag)$ define the state:

- we have a permutation of length i
- the last element is y
- $flag = \begin{cases} 0 & \text{if we end with an ascent} \\ 1 & \text{if we end with a descent} \end{cases}$

To compute, iterate through j , $flag'$ and if no constraints are violated, $dp(i, y, flag) += dp(i-1, j, flag')$

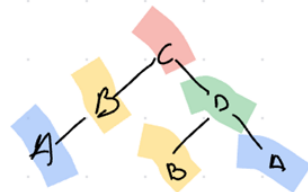
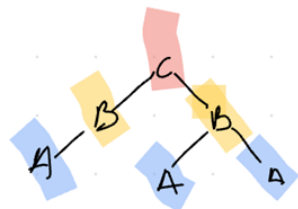
complexity is $O(n^3)$ and can be optimized by prefix sums to $O(n^2)$

G



Given is a tree whose nodes are associated values from 1 to k ($k=20$). Given is also a table of costs representing how much it costs to change a node's character. Determine the minimum possible cost to pay such that after changing the characters, all paths are non-palindromic.

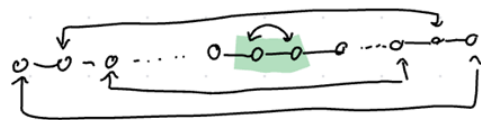
G



ODD:



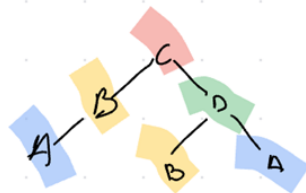
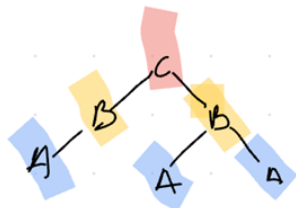
EVEN:



Given is a tree whose nodes are associated values from 1 to k ($k=20$). Given is also a table of costs representing how much it costs to change a node's character. Determine the minimum possible cost to pay such that uterious to changing the characters, all paths are non-palindromic.

First observation: A necessary and sufficient condition is to disallow length 2 and 3 palindromic paths

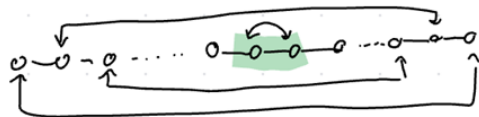
G



ODD:



EVEN:

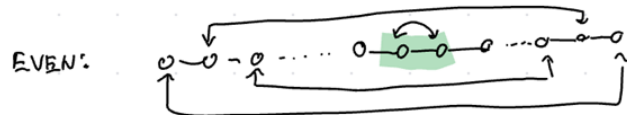
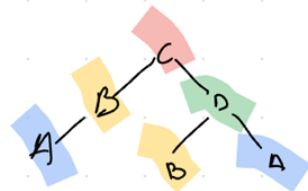
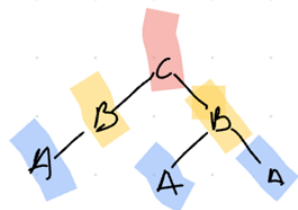


First observation: A necessary and sufficient condition is to disallow length 2 and 3 palindromic paths

A dp that exploits this would be the following:

$dp_{NODE, X, Y}$ → cost to make NODE's subtree correct, NODE's value to be X , and restrict the parent (if any) of NODE to be equal to Y

G



First observation: A necessary and sufficient condition is to disallow length 2 and 3 palindromic paths

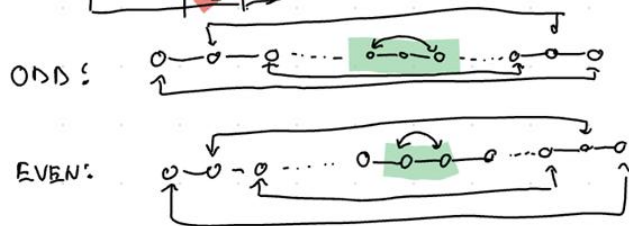
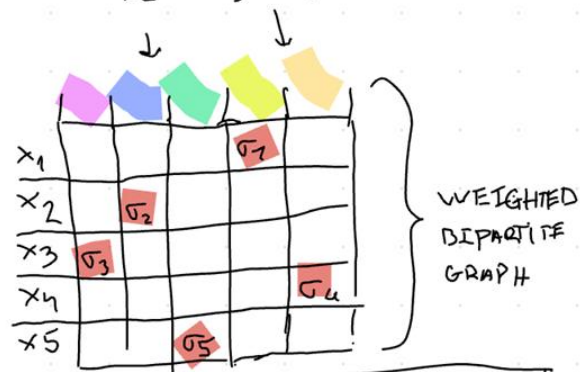
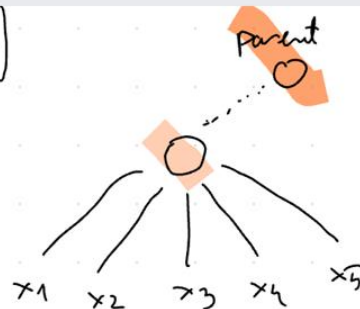
A dp that exploits this would be the following:

$dp_{NODE, x, y}$ $\rightarrow$ cost to make NODE's subtree correct, NODE's value to be x , and restrict the parent (if any) of NODE to be equal to y

The recurrence would look like

$$dp_{NODE, x, y} = \min \left(\sum_{\sigma} dp_{SON, \sigma_i, x} \mid \sigma \text{ distinct assignment} \right) + \text{cost}(\text{NODE} \rightarrow x)$$

G



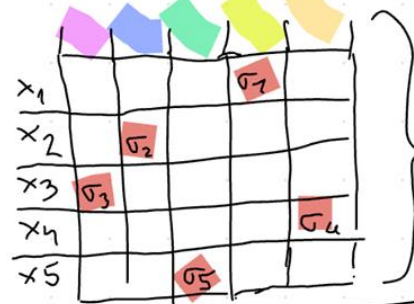
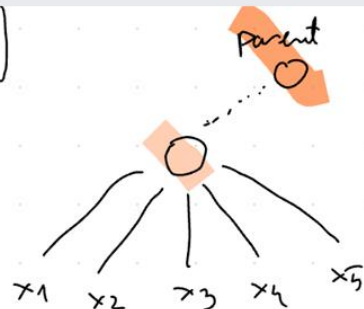
$dp_{NODE, x, y}$ $\rightarrow$ cost to make NODE's subtree correct, NODE's value to be x , and restrict the parent (if any) of NODE to be equal to y

The recurrence would look like

$$dp_{NODE, x, y} = \min \left(\sum_{\sigma} dp_{SON, \sigma_i, x} \mid \sigma \text{ distinct assignment} \right) + \text{cost}(NODE \rightarrow x) \quad !(\sigma_i \neq x, \sigma_i \neq y)!$$

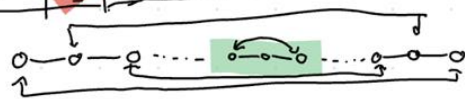
This can be calculated by fixing x and y and calculating max cost matching

G

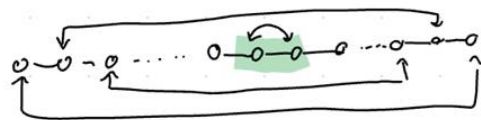


WEIGHTED
BIPARTITE
GRAPH

ODD:



EVEN:



$dp_{NODE, x, y} \rightarrow$ cost to make NODE's subtree correct, NODE's value to be x , and restrict the parent (if any) of NODE to be equal to y

The recurrence would look like

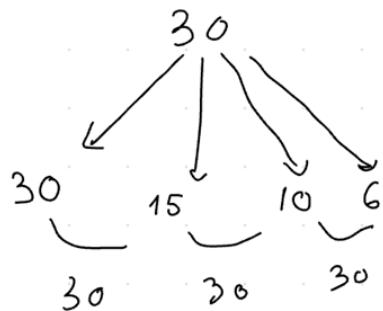
$$dp_{NODE, x, y} = \min \left(\sum_{\sigma} dp_{SON, \sigma_i, x} \mid \sigma \text{ distinct assignment} \right) + \text{cost}(NODE \rightarrow x) \quad !(\sigma_i \neq x, \sigma_i \neq y)!$$

This can be calculated by fixing x and y and calculating max cost matching

Final complexity should be around $O(k^3) \times O(N \cdot k^2)$ (if using hungarian algorithm) (But MCMF should pass as well)

(unsatisfyingly, all attempts to optimize this dp have failed, I think because it's not a subproblem)

4

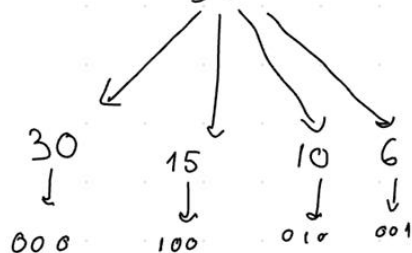


Given a number N , determine one of the largest sets of numbers such that any two numbers in the set have an LCM equal to N .

(H)

$$2 \times 3 \times 5$$

30



Given a number N , determine one of the largest sets of numbers such that any two numbers in the set have an LCM equal to N .

Consider the optimal solution for an N to be S_N .

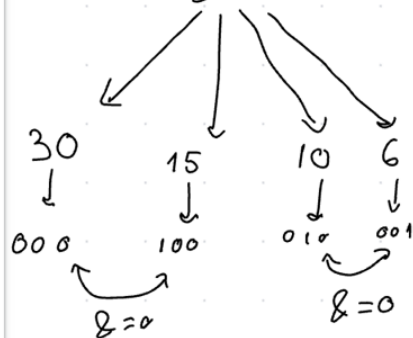
Let M_N be the set of bitmarks obtainable by the following procedure:

$$M_N = \{ b_x \mid (b_x)_i = \begin{cases} 0, & \text{if } i \text{ is not prime} \vee i \nmid N \vee i \neq 1 \\ 1, & \text{if the contrary} \end{cases} \text{ for } x \in S_N \}$$

H

$$2 \times 3 \times 5$$

30



Consider the optimal solution for an N to be S_N .

Let M_N be the set of bitmasks obtainable by the following procedure:

$$M_N = \{ b_x \mid (b_x)_i = \begin{cases} 0, & \text{if } i \text{ not prime } \vee i \nmid N \vee i \nmid x \\ 1, & \text{if the contrary} \end{cases}$$

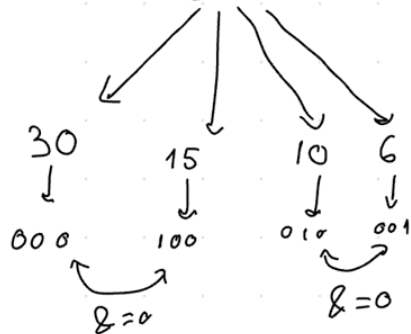
$$\text{for } x \in S_N \}$$

Take any two bitmasks $x, y \in M_N \Rightarrow x \& y = 0$
 $\Rightarrow$ each bit can appear at most once
 according to the rules, $\max \# \text{bits} = \# \text{prime factors}(N)$
 So $|S_N| \leq \# \text{prime factors}(N) + 1$

H

$$2 \times 3 \times 5$$

30



Consider the optimal solution for an N to be S_N .

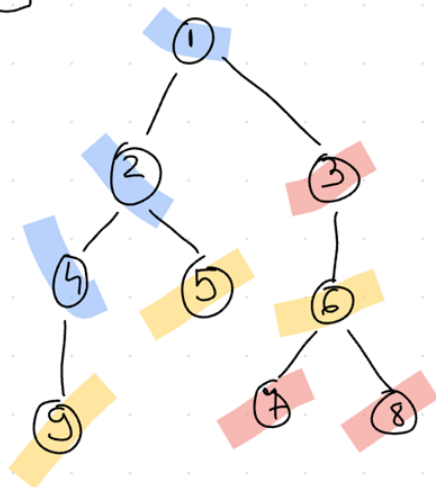
Let M_N be the set of bitmasks obtainable by the following procedure:

$$M_N = \{ b_x \mid (b_x)_i = \begin{cases} 0, & \text{if } i \text{ not prime } \vee i \nmid N \vee i \neq x \\ 1, & \text{if the contrary} \end{cases} \text{ for } x \in S_N \}$$

Take any two bitmasks $x, y \in M_N \Rightarrow x \& y = 0$
 $\Rightarrow$ each bit can appear at most once
according to the rules, $\max \# \text{ bits} = \# \text{ prime factors}(N)$
So $|S_N| \leq \# \text{ prime factors}(N) + 1$

One such $S_N = \left(\frac{N}{x} \mid x=1 \vee x \mid N \text{ and } x \text{ is prime} \right)$

I

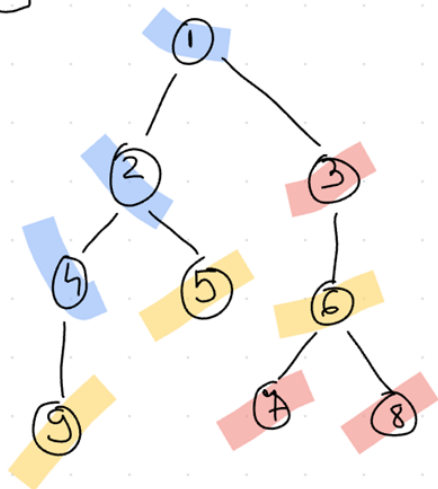


For a tree where each node is associated a color, select some chains such that every pair of endpoints consists of nodes of equal color, and every color has exactly 1 chain whose endpoints are colored in it.

Let then a_x be the count of chains in which x appears.

Determine the maximum lexicographic array d that can be obtained through this procedure.

I



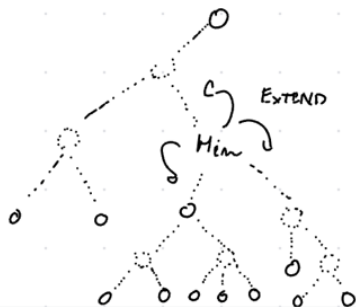
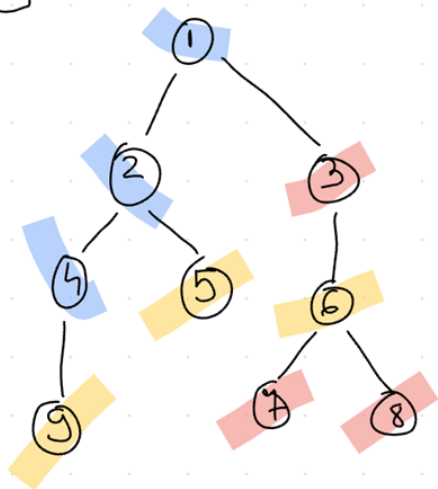
For a tree where each node is associated a color, select some chains such that every pair of endpoints consists of nodes of equal color, and every color has exactly 1 chain whose endpoints are colored in it.

Let then a_x be the count of chains in which x appears.

Determine the maximum lexicographic array d that can be obtained through this procedure.

First possible solution: virtual tree span.

I

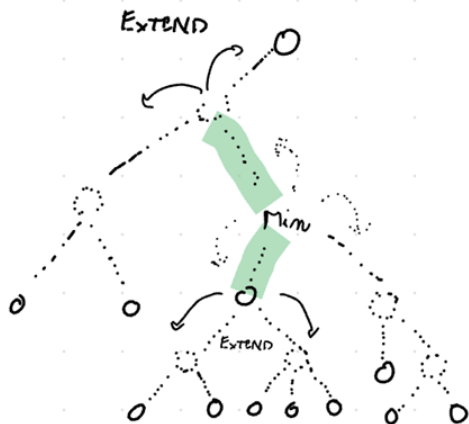


For a tree where each node is associated a color, select some chains such that every pair of endpoints consists of nodes of equal color, and every color has exactly 1 chain whose endpoints are colored in it.

Let then d_x be the count of chains in which x appears.

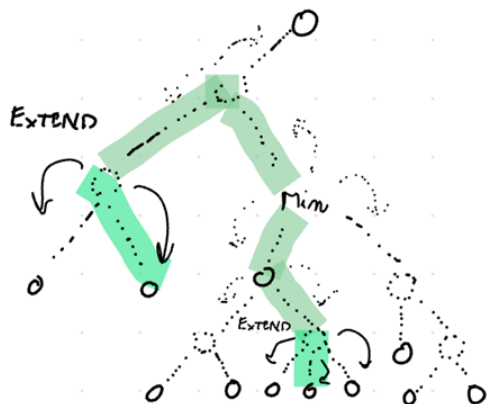
Determine the maximum lexicographic array d that can be obtained through this procedure.

First possible solution: Virtual tree space.
After computing the virtual, find the minimum node that appears in any chain



First possible solution: Virtual tree span.
After computing the virtual, find the minimum node that appears in any chain $\rightarrow$ Add it to the structure

$\rightarrow$ Start developing the chain from the minimum onwards (by choosing the virtual edge with the smallest different minimum)



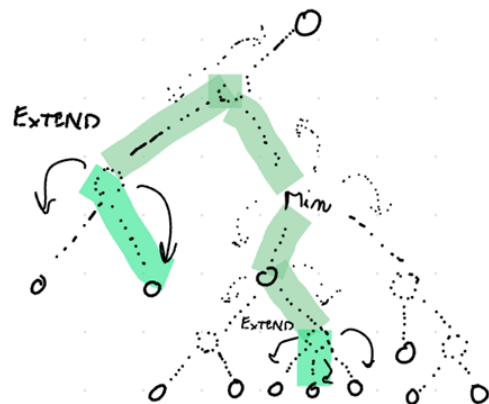
First possible solution: Virtual tree span.
After computing the virtual, find the minimum node that appears in any chain $\rightarrow$ Add it to the structure

$\rightarrow$ Start developing the chain from the minimum onwards (by choosing the virtual edge with the smallest different minimum)

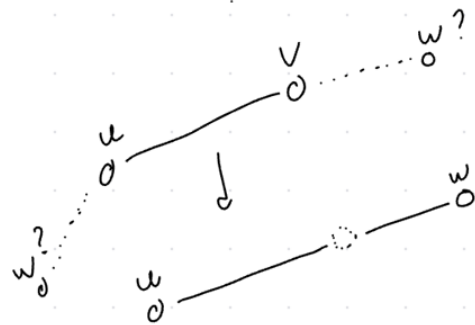
$\hookrightarrow$ End process when you reach a leaf



Second solution (cooler): greedy



I



Second solution (cooler): greedy

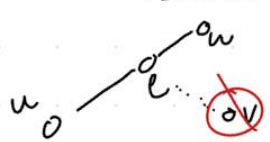
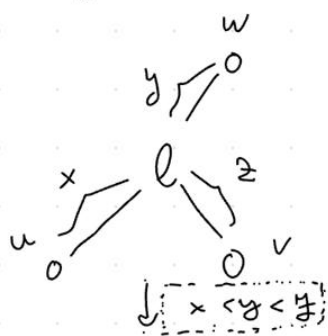
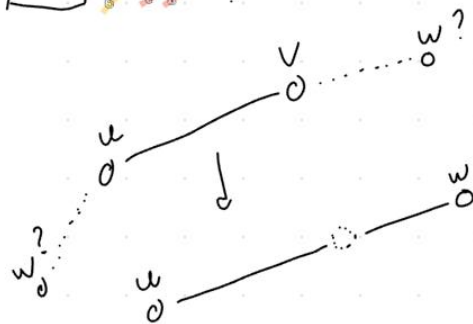
↳ start with any chain; perform the following exchanges for every node that remains for a color

(if current chain is $u-v$ and we consider w)

I u, v and w are colinear

continue with the longer chain formed by any two

I



Second solution (cooler): greedy

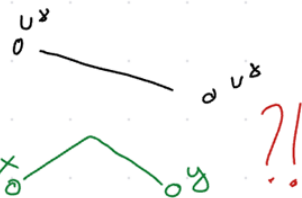
start with any chain; perform the following exchanges for every node that remains for a color

(if current chain is $u-v$ and we consider w)

I u, v and w are colinear
continue with the longer chain formed by any two

II u, v and w are not colinear
let l be their common point
take the endpoints that correspond to the lowest two minimums on the path from l to them

I

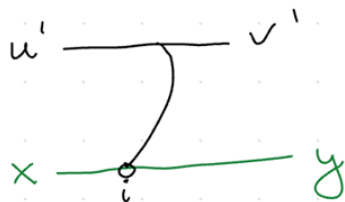
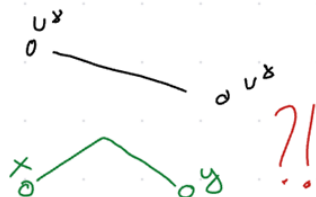


I u, v and w are colinear
continue with the longer chain formed
by any two

II u, v and w are not colinear
let l be their common point
take the endpoints that correspond to the
lowest two minimums on the path from l to them

Assume greedy is wrong, optimal (and provably
unique) chain is $x-y$, we obtained something
else instead.

I



Assume greedy is wrong, optimal (and provably unique) chain is $x-y$, we obtained something else instead.

Let $x < y$ so x is considered first.

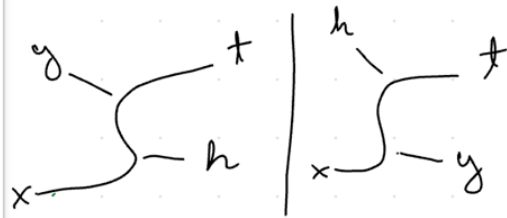
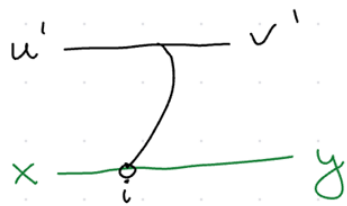
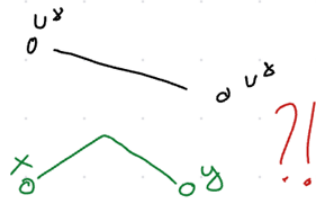
Let $u'-v'$ the chain when x is considered

Let i be the closest node of the $x-y$ path to the $u'-v'$ path

I First case: $\min(x-i) = \min(x-y)$

clearly $\min(x-y)$ is minimum over all possible chains, so x will be a part of the next chain.

I



Assume greedy is wrong, optimal (and provably unique) chain is $x-y$, we obtained something else instead.

Let $x < y$ so x is considered first.

Let $u'-v'$ the chain when x is considered

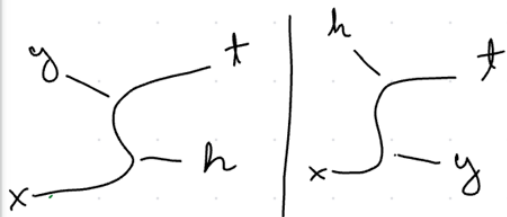
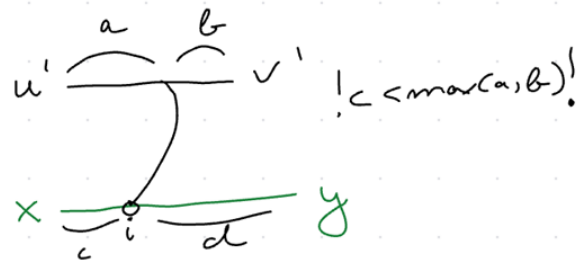
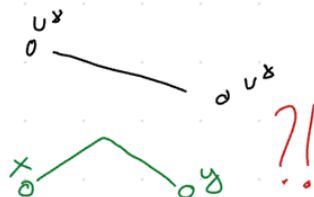
Let i be the closest node of the $x-y$ path to the $u'-v'$ path

I First case: $\min(x-i) = \min(x-y)$

clearly $\min(x-y)$ is minimum over all possible chains, so x will be a part of the next chain.

x will remain a part of the solution until the encounter with y and later because otherwise there exists a trivial extension for y to "greater optimality" (with the further node)

I



I First case: $\min(x-i) = \min(x-y)$

clearly $\min(x-y)$ is minimum over all possible chains, so x will be a part of the next chain.

x will remain a part of the solution until the encounter with y and later because otherwise there exists a trivial extension for y to "greater optimality" (with the further node)

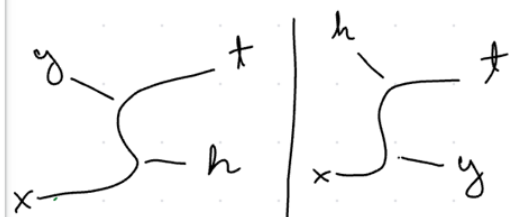
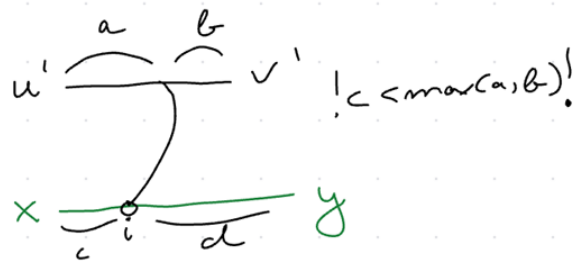
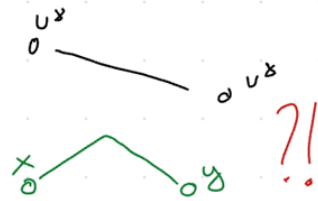
II Second case: $\min(x-i) \neq \min(x-y)$

if x is chosen, same as before

if not, same extension argument for y

So greedy will converge to optimum

I



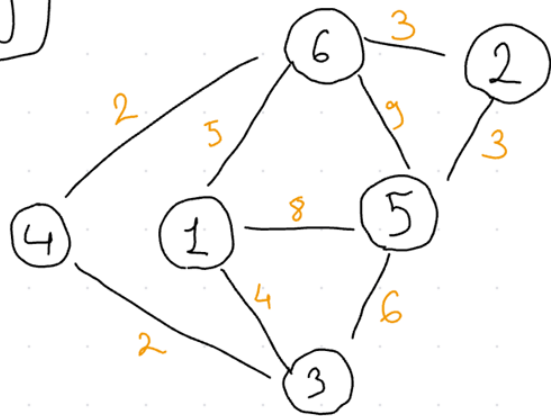
I First case: $\min(x-i) = \min(x-y)$
 clearly $\min(x-y)$ is minimum over all possible chains, so x will be a part of the next chain.
 x will remain a part of the solution until the encounter with y and later because otherwise there exists a trivial extension for y to a greater optimality (with the further node)

II Second case: $\min(x-i) \neq \min(x-y)$
 if x is chosen, same as before
 if not, same extension argument for y

So greedy will converge to optimum

Final complexity for both solutions: $O(N \times \log N)$

J



Given is a graph
with weighted edges

We have to answer queries
of the form:

$x \ y \ l$

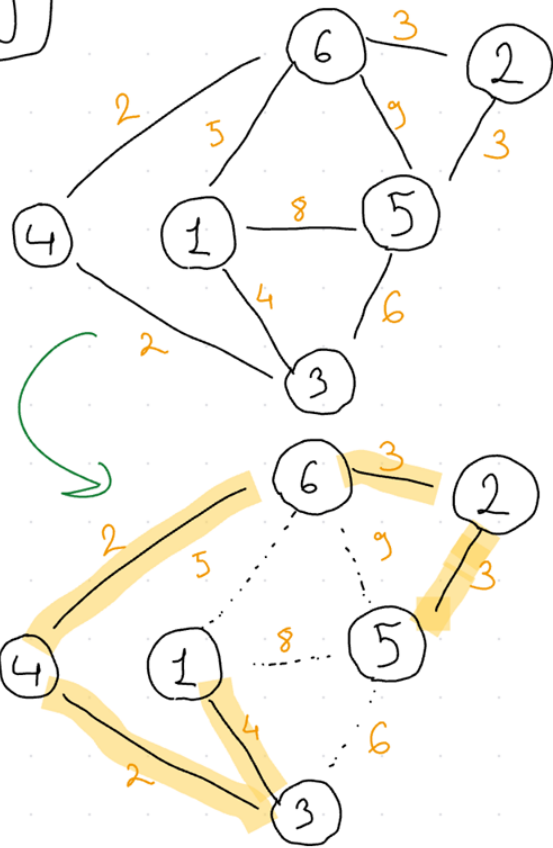
Meaning:

DETERMINE THE LOWEST COST OF A PATH
FROM x TO y THAT PASSES ONLY THROUGH NODES OF
DEGREE AT LEAST l

Intended time complexity:

$O((M+Q) \times \log N)$

J



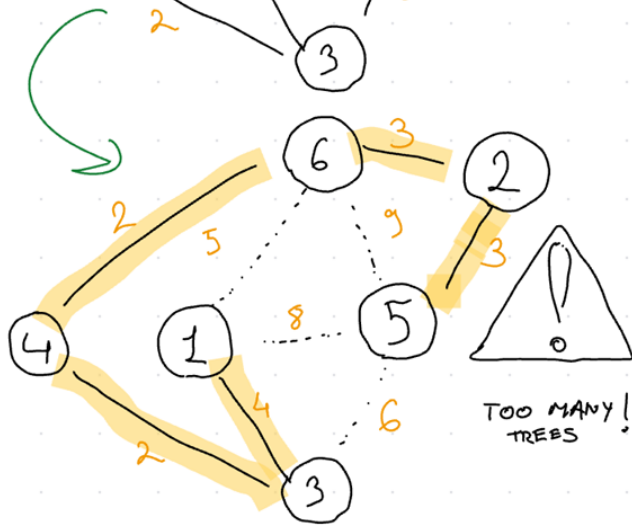
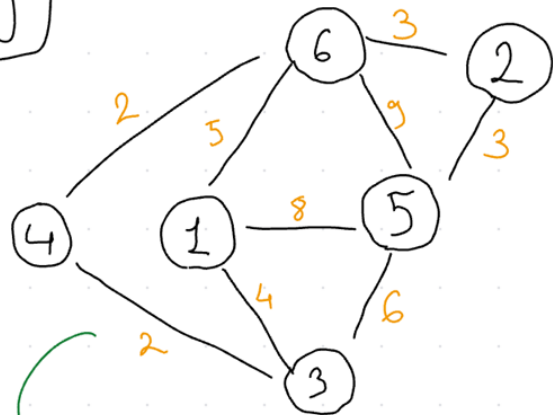
Virtual Seelback I: $l=1$ for all queries

Well known theoretical result: reducing the graph to its MST will not modify any mincost between any pair.

Mincost path under maximum is easier to calculate on an MST given uniqueness of the path

What remains is to be able to answer some path minimum in a computable tree (although we recommended to use methods such as KRT or Line Graph for these computations)

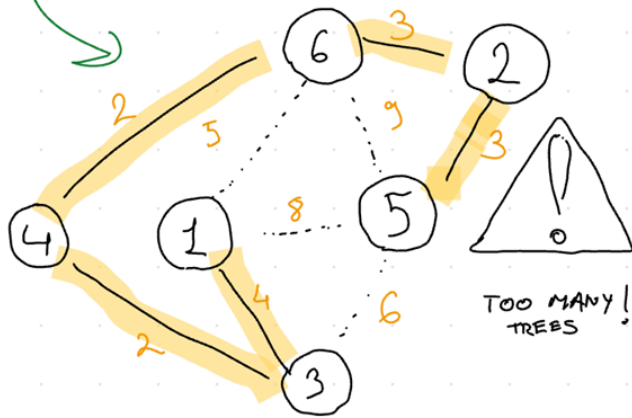
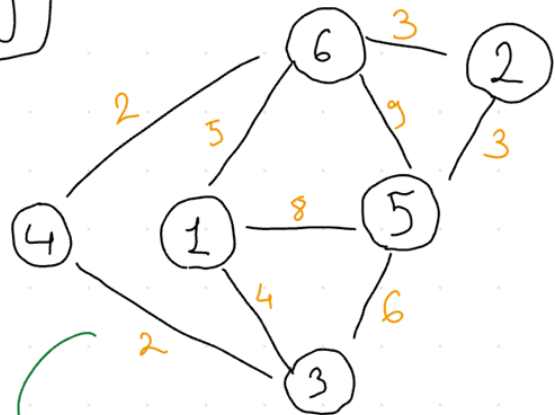
J



Virtual Seelbach II: N is small?

So just redo for every degree with
the relevant nodes and edges?

J

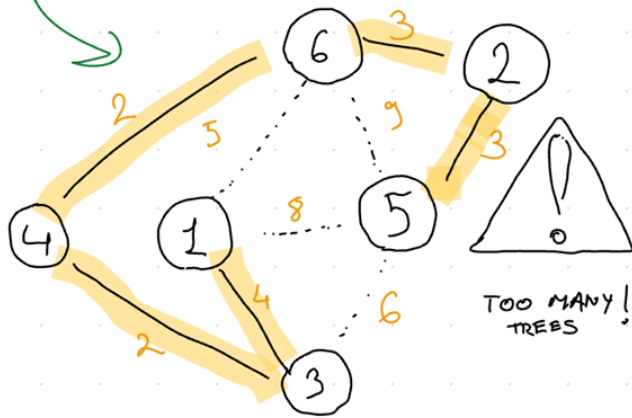
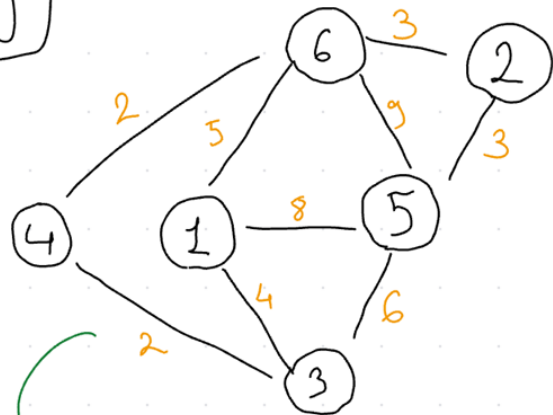


Virtual Seelbach II: N is small?

So just redo for every degree with
the relevant nodes and edges?

→ Probably $\sim O(N \cdot M)$

J



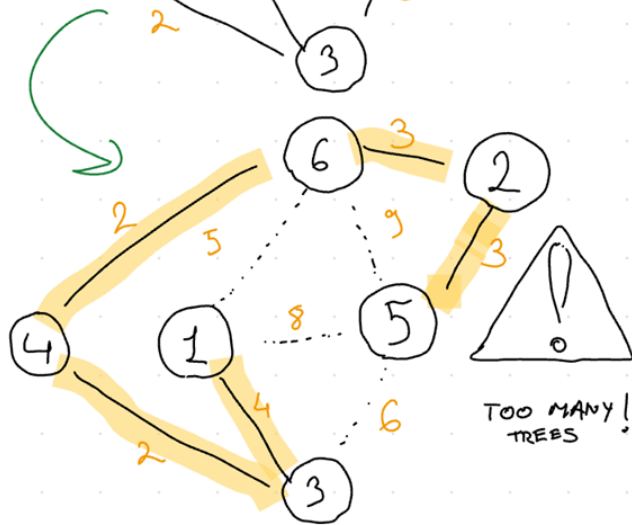
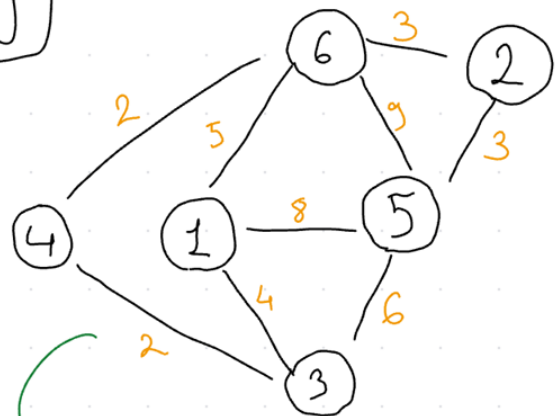
Virtual Seelbach II: N is small?

So just redo for every degree with
the relevant nodes and edges?

Probably $\sim O(N \cdot M)$

Unless...

J



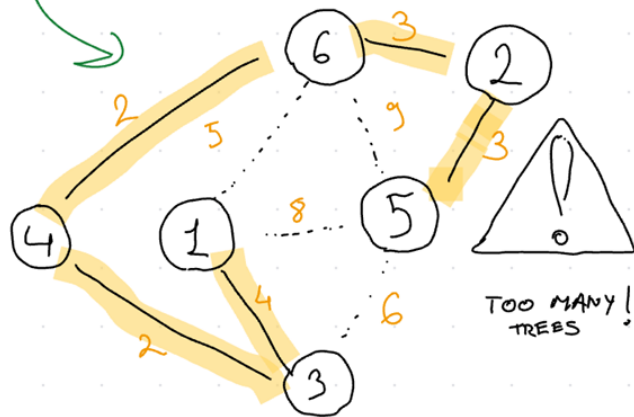
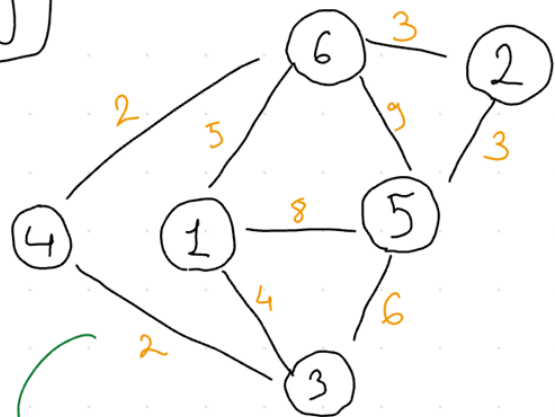
Virtual Seelbach II: N is small?

So just redo for every degree with the relevant nodes and edges?

Probably $\sim O(N \cdot M)$

Unless we notice that there are $O(\sqrt{N})$ possible distinct degrees!

J



Virtual Seelback II: N is small?

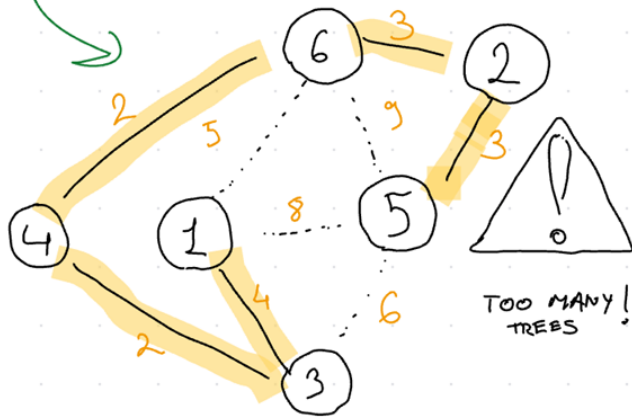
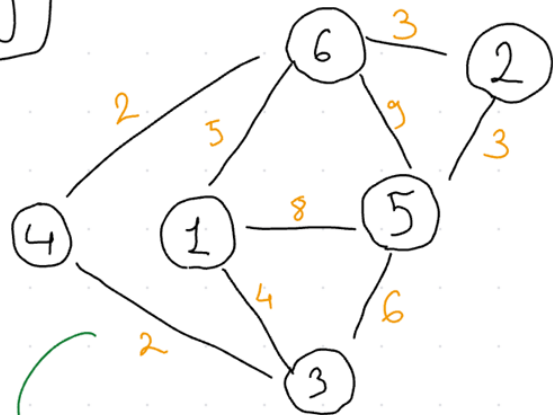
So just redo for every degree with the relevant nodes and edges?

Probably $\sim O(N \cdot M)$

Unless we notice that there are $O(\sqrt{M})$ possible distinct degrees!

Rebuilding the entire MST at each new degree would mean a total complexity of $O(\sqrt{M}) \times O(\text{build})$, and build is $\sim O((N+M) \log N)$, can be optimized to be almost linear

J



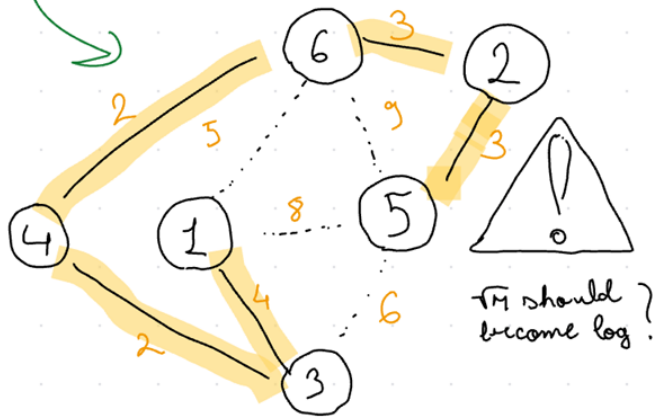
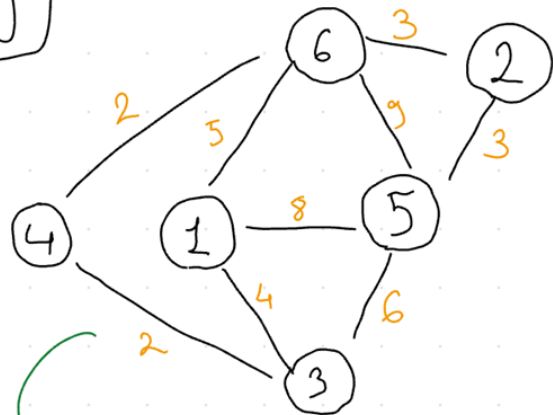
Virtual Seelabs II: N is small?

TLE

☹

(hopefully)

J



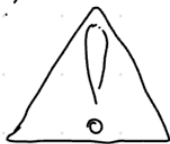
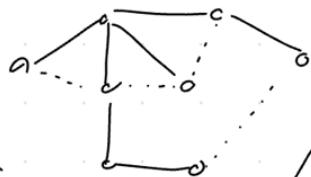
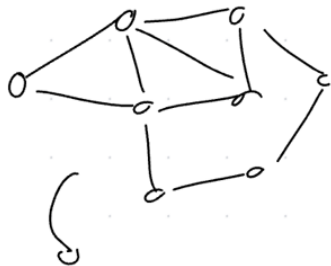
Virtual Seelback II: N is small?

So just redo for every degree with the relevant nodes and edges?

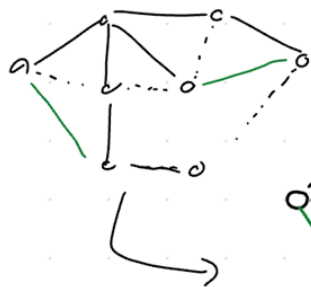
→ ~~Probably $O(N \cdot M)$~~
if smart, $\tilde{O}(M \sqrt{N})$

There is still a lot of redundancy

J



TM should become log?



Virtual Seelback II: N is small?

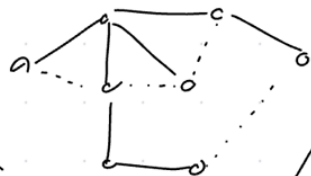
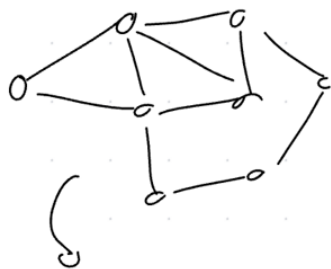
So just redo for every degree with the relevant nodes and edges?

~~Probably $O(N \cdot M)$~~
if smart, $\tilde{O}(M \sqrt{N})$

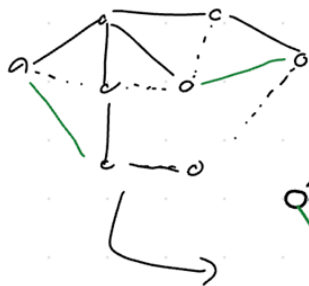
There is still a lot of redundancy

Consider "THE FIRST EVER" MST. Now the edges that didn't make the cut. Will they make the cut as we add more edges?

J



TM should become log!



Virtual Seelbuck II: N is small?

So just redo for every degree with the relevant nodes and edges?

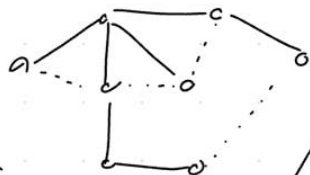
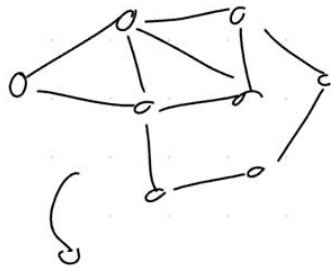
~~Probably $O(N \cdot M)$~~
if smart, $\tilde{O}(M \sqrt{N})$

There is still a lot of redundancy

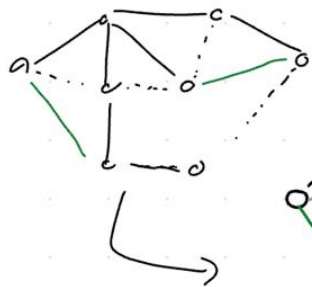
Consider "THE FIRST EVER" MST. Now the edges that didn't make the cut. Will they make the cut as we add more edges?

No!

J



TM should become log.



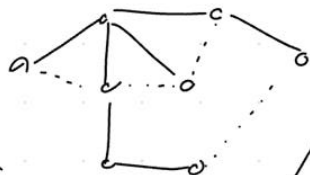
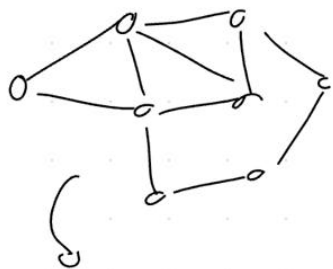
Virtual Seelback II: N is small?

There is still a lot of redundancy

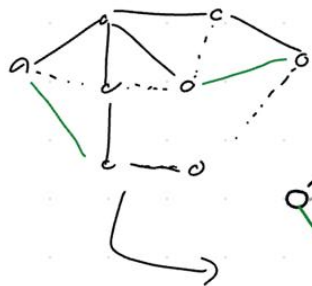
Consider "THE FIRST EVER" MST. Now the edges that didn't make the cut. Will they make the cut as we add more edges? $\rightarrow$ **No**

So, then, when we do a new MST for some new degree, we can set the graph to be MST'd to be $[(\text{the old MST}) \cup (\text{the new edges})]$

J



TM should become log.



Virtual Seelback II: N is small?

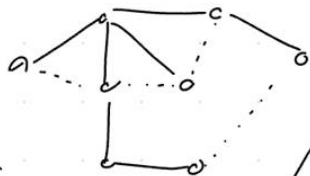
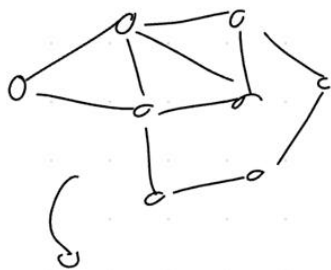
There is still a lot of redundancy

Consider "THE FIRST EVER" MST. Now the edges that didn't make the cut. Will they make the cut as we add more edges? $\rightarrow$ **No**

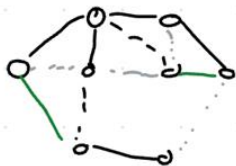
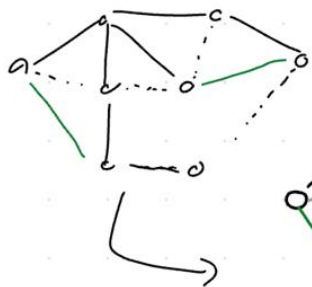
So, then, when we do a new MST for some new degree, we can set the graph to be MST'd to be $[(\text{the old MST}) \cup (\text{the new edges})]$

Claim: with this new optimisation in mind, the total amount of edges sent to the building of some MST is $O(M)$.

J



TM should become $O(M)$?



Virtual Seelback II: N is small?

There is still a lot of redundancy

So, then, when we do a new MST for some new degree, we can set the graph to be MST'd to be $(\text{the old MST}) \cup (\text{the new edges})$

Claim: with this new optimisation in mind, the total amount of edges sent to the building of some MST is $O(M)$.

Clearly, complexity is $O(Q \cdot \log N) +$

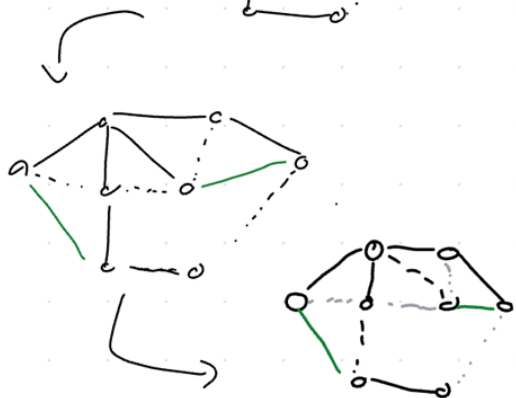
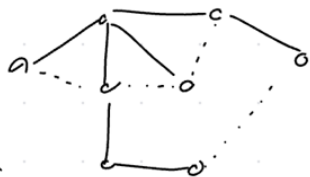
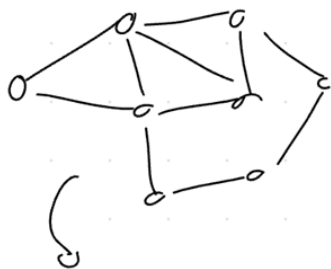
$O(\sum_{d=1}^N \text{BUICK}(E_d))$, where $E_d = \text{MST for deg} \geq d$

1 # edges with 1 endpoint with $\text{deg} = d$

And $\text{BUICK}(x) = O(x \log x)$

So we want $O(\sum E_d) = O(M)$

J



Virtual Seelback II: N is small?

There is still a lot of redundancy

So, then, when we do a new MST for some new degree, we can set the graph to be MST'd to be $(\text{the old MST}) \cup (\text{the new edges})$

Clearly, complexity is $O(Q \cdot \log N) + O(\sum_{d=1}^N \text{BUILD}(E_d))$, where $E_d = |\text{MST for deg} \geq d| + \underbrace{|\# \text{ edges with endpoint with deg} = d|}_{c_d}$

And $\text{BUILD}(x) = O(x \log x)$

So we want $O(\sum E_d) = O(M)$

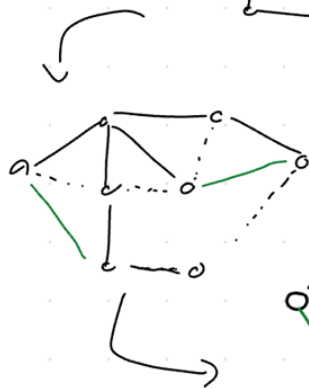
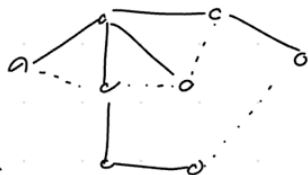
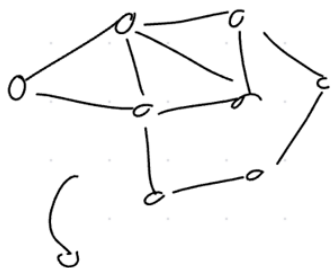
if $E_d = |\text{MST}(d+1)| + c_d$, $\sum c_d \leq 2M$

so $\sum |\text{MST}(d+1)| = O(M)$

$|\text{MST}(d+1)| = \#(\text{Nodes of degree} > d) - 1$

$\sum_{d=1}^N \#(\text{Nodes w/ deg} > d) - 1 = \sum_{d=1}^N \sum_{u \in V} [\text{deg}(u) > d]$

J



(swap order)

Virtual Seelback II: N is small?

There is still a lot of redundancy

So, then, when we do a new MST for some new degree, we can set the graph to be MST'd to be $[(\text{the old MST}) \cup (\text{the new edges})]$

So we want $O(\sum E_d) = O(M)$

if $E_d = |\text{MST}(d+1)| + c_d$, $\sum c_d \leq 2M$

so $\# \sum |\text{MST}(d+1)| = O(M)$

$|\text{MST}(d+1)| = \#(\text{Nodes of degree} > d) - 1$

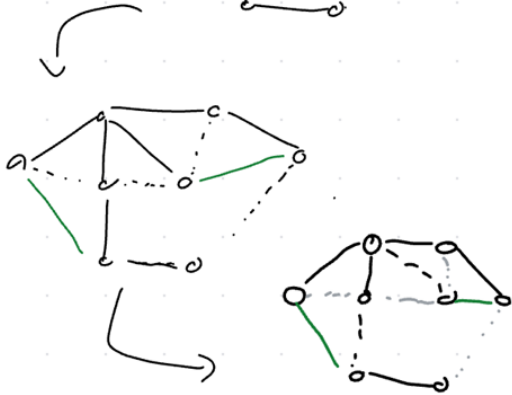
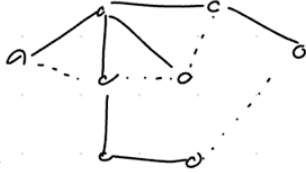
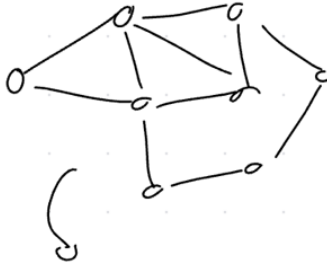
$$\sum_d \#(\text{Nodes w/ deg} > d) - 1 = \sum_d \sum_{u \in V} [\deg(u) > d]$$

$$= \sum_{u \in V} \sum_d [\deg(u) > d] = \sum_{u \in V} \deg(u) = 2M = O(M)$$

So $O(\sum E_d) = O(M)$, then $O(\sum_{\text{query}} E_d) = O(M \log N)$

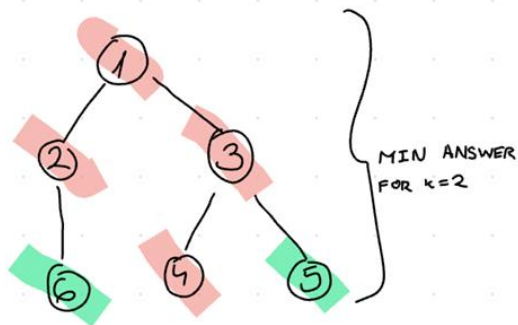
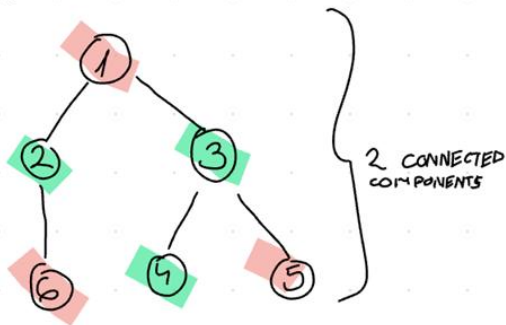
(BUILD and answering queries can be optimized to $O(\alpha(x))$)

J



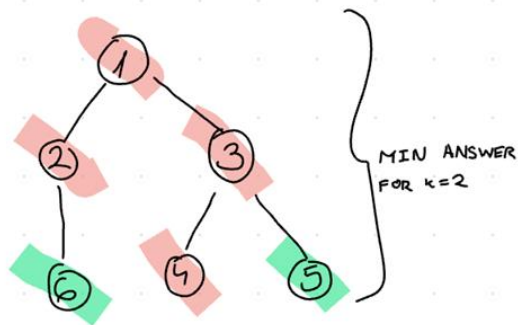
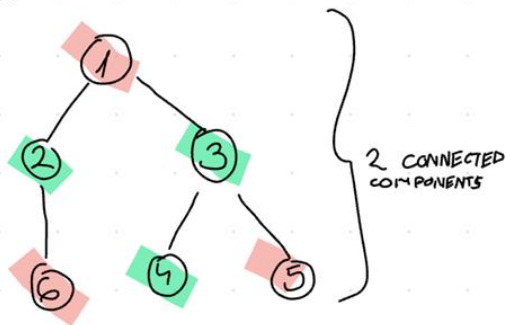
~~Virtual Seabash II~~: NID! 2

K



Given an unrooted tree, determine a binary string (of length N) such that the induced graph of the nodes v such that $S_v=1$ has exactly K connected components

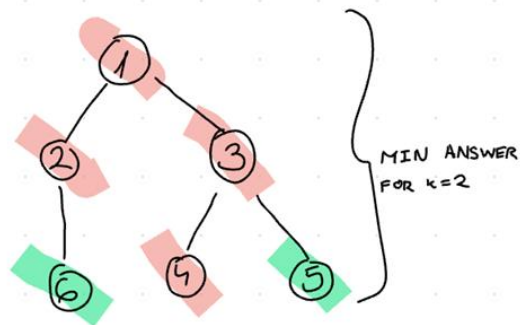
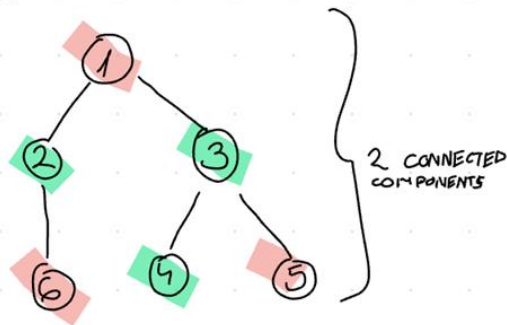
K



Given an unrooted tree, determine a binary string (of length N) such that the induced graph of the nodes v such that $S_v=1$ has exactly K connected components

We need a way to calculate connected components that is canonic with the standard min-lex framework

K



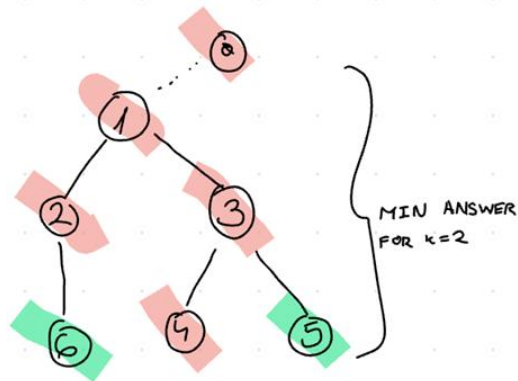
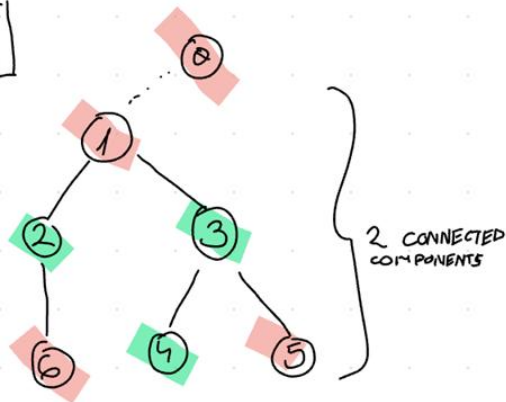
Given an unrooted tree, determine a binary string (of length N) such that the induced graph of the nodes v such that $S_v = 1$ has exactly K connected components

We need a way to calculate connected components that is canonic with the standard min-lex framework

→ DSU will not be useful

→ But we are working on trees, so there must be something easier...

K

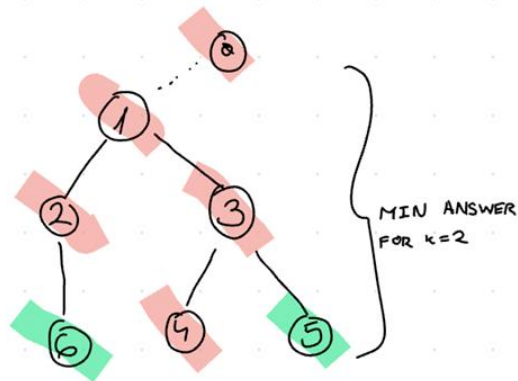
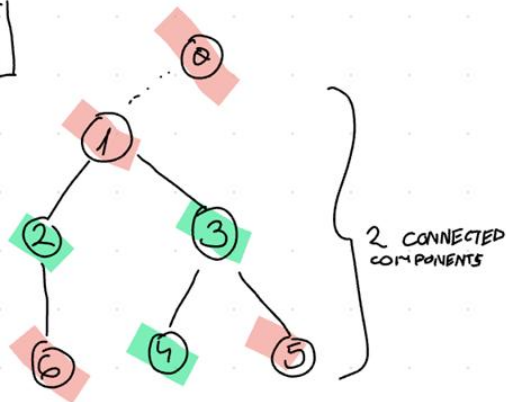


Given an unrooted tree, determine a binary string (of length N) such that the induced graph of the nodes v such that $S_v = 1$ has exactly K connected components

Every connected component has a "root" for an arbitrary rooting

↳ Count of cc's is the number of distinct roots

K



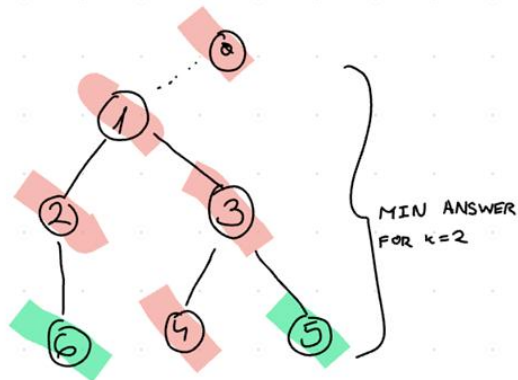
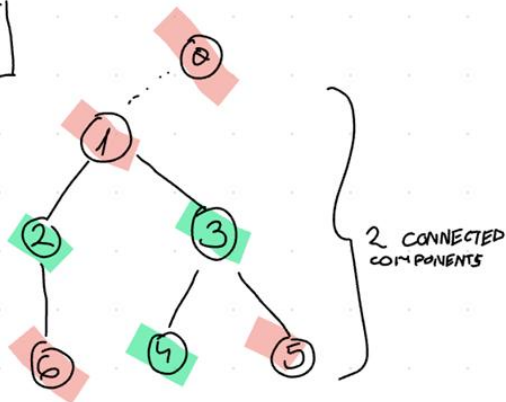
Given an unrooted tree, determine a binary string (of length N) such that the induced graph of the nodes v such that $S_v=1$ has exactly K connected components

Every connected component has a "root" for an arbitrary rooting

↳ Count of cc's is the number of distinct roots

↳ A root is an active node whose father is inactive

K



Given an unrooted tree, determine a binary string (of length N) such that the induced graph of the nodes v such that $S_v = 1$ has exactly K connected components

Every connected component has a "root" for an arbitrary rooting

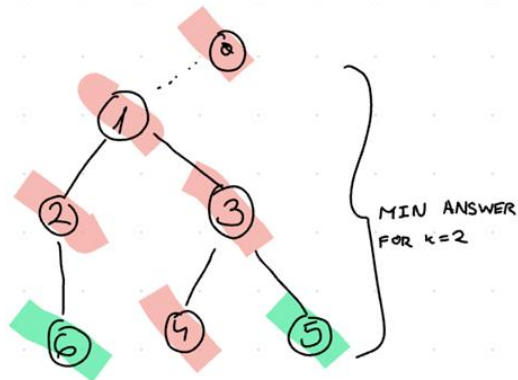
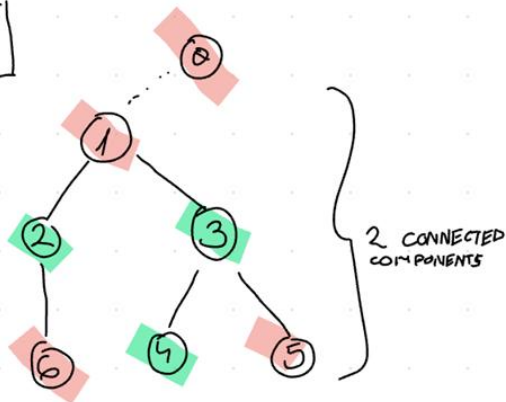
↳ Count of cc's is the number of distinct roots

↳ A root is an active node whose father is inactive

Consider the optimal solution T

If there is a node v such that $T_v = T_{P_v} = 1$, $T'_i = T_i \forall i \neq v$ and $T'_v = 0$ is "more optimal"

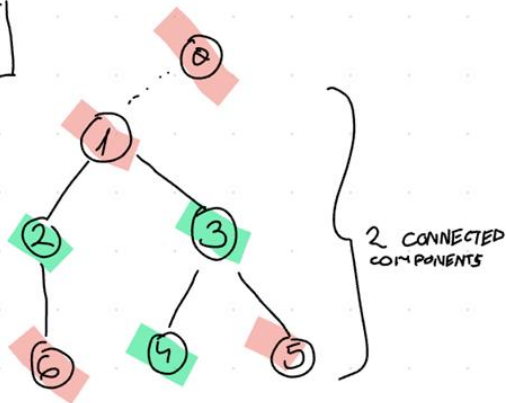
K



Given an unrooted tree, determine a binary string (of length N) such that the induced graph of the nodes v such that $S_v=1$ has exactly K connected components

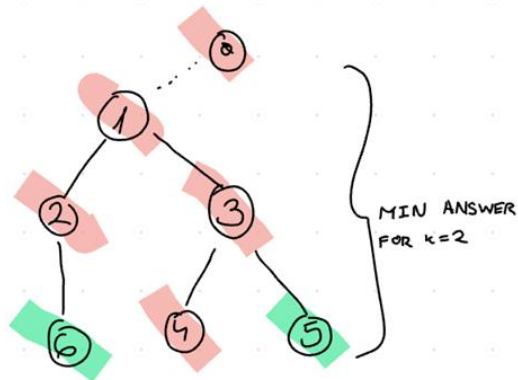
- A [A root is an active node
whose father is inactive
- B [Consider the optimal solution T
If there is a node v such that $T_v = T_{P_v} = 1$,
 $T'_i = T_i \forall i \neq v$ and $T'_v = 0$ is "more optimal"
- So according to A and B (but mostly B),
the final solution will describe an
INDEPENDENT SET

K

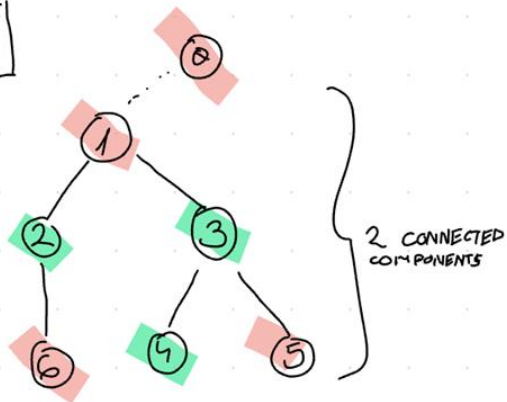


NEW
STATEMENT

Given an unrooted tree, determine a binary string (of length N) such that $\{v | S_v = 1\}$ describes an INDEPENDENT SET of size K



K

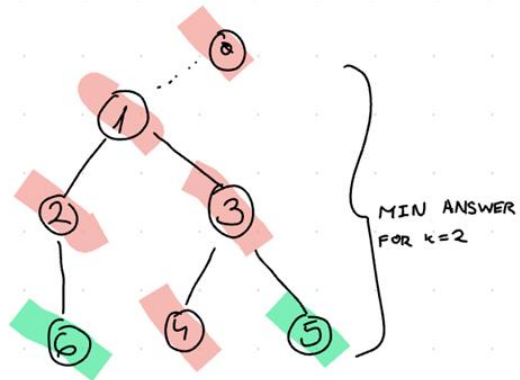


NEW
STATEMENT

Given an unrooted tree, determine a binary string (of length N) such that $\{v | S_v = 1\}$ describes an INDEPENDENT SET of size K

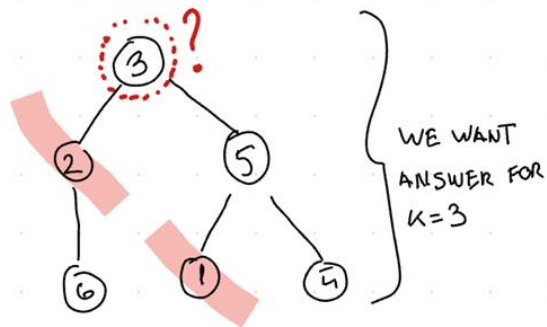
Standard mex framework: set a value to something, check it still in constraints;

The constraint: we can still select some more nodes to create an iset of size k



→ We can check this by calculating the maximum iset that can be formed with the rest of the nodes

K



Standard mex framework: set a value to something, check it still in constraints;

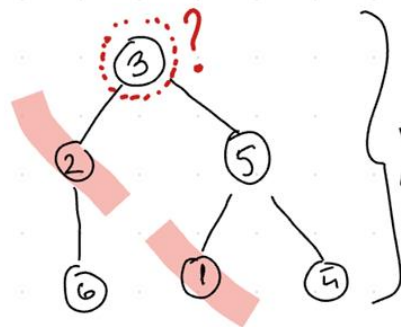
The constraint: we can still select some more nodes to create an. iset of size k

→ We can check this by calculating the maximum iset that can be formed with the rest of the nodes

The standard dp for this is

$$dp_{node, 0/1} \rightarrow \begin{cases} dp_{node, 0} \rightarrow \sum \max(dp_{x, 0}, dp_{x, 1}) \\ dp_{node, 1} \rightarrow \sum dp_{x, 1} \end{cases} \quad x \in \text{children}(node)$$

K



WE WANT
ANSWER FOR
K=3

Standard n/w lex framework: set a value to something, check it still in constraints;
The constraint: we can still select some more nodes to create an. iset of size k

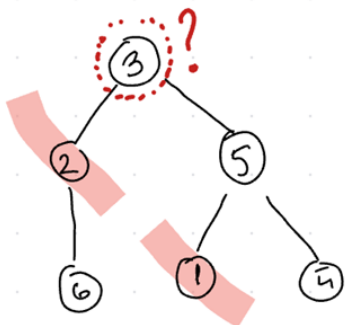
→ We can check this by calculating the maximum iset that can be formed with the rest of the nodes

The standard dp for this is

$$dp_{node, 0/1} \rightarrow \begin{cases} dp_{node, 0} \rightarrow \sum_{x \in C(node)} \max(dp_{x, 0}, dp_{x, 1}) + A_{node} \\ dp_{node, 1} \rightarrow \sum_{x \in D(node)} dp_{x, 1} + B_{node} \end{cases}$$

usually dp to add constraints

K



WE WANT
ANSWER FOR
K=3

The standard dp for this is

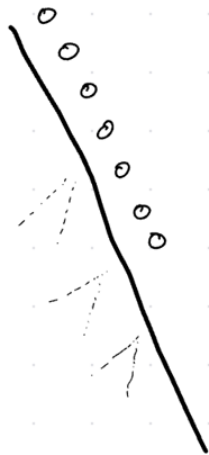
$$dp_{node, 0/1} \rightarrow \begin{cases} dp_{node, 0} \rightarrow \sum_{x \in S(node)} \max(dp_{x, 0}, dp_{x, 1}) + A_{node} \\ dp_{node, 1} \rightarrow \sum_{x \in D(node)} dp_{x, 1} + B_{node} \end{cases}$$

Validity dp to avoid constraints

We want the root value of this dp subject to online updates of A_{node} and B_{node} .

For this, we can use HLD.

K



The standard dp for this is

$$dp_{node, 0/1} \rightarrow \begin{cases} dp_{node, 0} \rightarrow \sum_{x \in S(node)} \max(dp_{x, 0}, dp_{x, 1}) + A_{node} \\ dp_{node, 1} \rightarrow \sum_{x \in D(node)} dp_{x, 1} + B_{node} \end{cases}$$

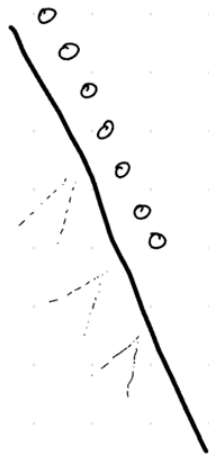
Ability dp to add constraints

We want the root value of this dp subject to online updates of A_{node} and B_{node} .

For this, we can use HLD.

Consider the different runs gained from light nodes. We integrate them into the A_{node} and B_{node} respectively.

K



The standard dp for this is

$$\begin{aligned} dp_{node, 0/1} &\rightarrow \begin{cases} dp_{node, 0} \rightarrow \sum \max(dp_{x, 0}, dp_{x, 1}) + A_{node} \\ dp_{node, 1} \rightarrow \sum dp_{x, 1} + B_{node} \end{cases} \\ &\quad \text{validity dp to add constraints} \end{aligned}$$

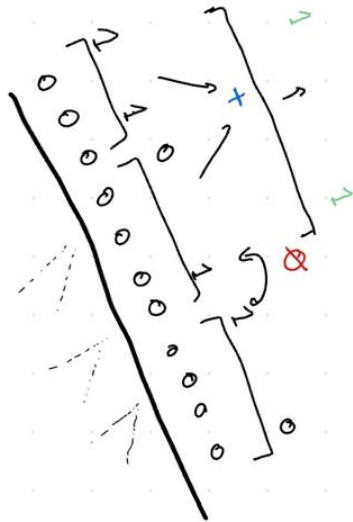
For this, we can use HLD

Consider the different runs gained from light nodes. We integrate them into the A_{node} and B_{node} respectively.

Now, we have to integrate the recurrence to be able to work on a line (and a segment tree)

We store for a segment $dp(c/l)(o/i)$ the maximum value of the dp when the left/right end is included or not in the solution

K



The standard dp for this is

$$\begin{aligned} dp_{node, 0/1} &\rightarrow \begin{cases} dp_{node, 0} \rightarrow \sum \max(dp_{x, 0}, dp_{x, 1}) + A_{node} \\ dp_{node, 1} \rightarrow \sum dp_{x, 1} + B_{node} \end{cases} \end{aligned}$$

Validity dp to avoid constraints

For this, we can use HLD

Now, we have to integrate the recurrence to be able to work on a line (and a segment tree)

We store for a segment $dp(c/l)(o/i)$ the maximum value of the dp when the left/right end is included or not in the solution

We merge states that respect the criteria that an iset is in fact independent.

Final complexity : $O(N \times \log^2 N)$